

National College of Ireland

Project Submission Sheet

Student Name: Surya Chandra Raju Kurapati
Student ID: x23396920
Programme: MSc in Data Analytics (MSCDAD_B) **Year:** 2025-2026
Module: Domain Applications
Lecturer: Vikas Sahni
Submission Due Date: 20-Nov-2025
Project Title: RadAssistIQ – Intelligent Chest X-Ray Diagnostic Assistant to Read Smarter and Diagnose Faster

Word Count: 2631

I hereby certify that the information contained in this (my submission) is information pertaining to research I conducted for this project. All information other than my own contribution will be fully referenced and listed in the relevant bibliography section at the rear of the project.

Signature: Surya Chandra Raju Kurapati
Date: 20-Nov-2025

PLEASE READ THE FOLLOWING INSTRUCTIONS:

1. Please attach a completed copy of this sheet to each project (including multiple copies).
2. Projects should be submitted to your Programme Coordinator.
3. **You must ensure that you retain a HARD COPY of ALL projects**, both for your own reference and in case a project is lost or mislaid. It is not sufficient to keep a copy on computer. Please do not bind projects or place in covers unless specifically requested.
4. You must ensure that all projects are submitted to your Programme Coordinator on or before the required submission date. **Late submissions will incur penalties.**
5. All projects must be submitted and passed in order to successfully complete the year. **Any project/assignment not submitted will be marked as a fail.**

Office Use Only

Signature:

Date:

Penalty Applied (if applicable):

AI Acknowledgement Supplement

Domain Applications

RadAssistIQ – Intelligent Chest X-Ray Diagnostic Assistant to Read Smarter and Diagnose Faster

Your Name/Student Number	Course	Date
Surya Chandra Raju Kurapati	MSc in Data Analytics	20-Nov -2025

AI Acknowledgment

Tool Name	Brief Description	Link to tool
ChatGPT	Used to understand errors while integrating DenseNet121 integration with python and Fast API and key concepts of Radiology department process and chest x-ray scanning procedures.	https://chatgpt.com/

Description of AI Usage

This section provides a more detailed description of how the AI tools were used in the assignment. It includes information about the prompts given to the AI tool, the responses received, and how these responses were utilized or modified in the assignment. **One table should be used for each tool used.**

ChatGPT	
While implementing the Python code for RadAssistIQ, I encountered several errors along the way, but I was able to identify and resolve them with the help of ChatGPT	
Error: RequestResponseCycle.receive of Validate for below causes: <uvicorn.protocols.http.h11_impl.RequestResponseCycle1. Blocking CPU work inside an async def object at 0x146673640>> L {'type': 'http', 'asgi':2. Name collision / duplicate route definitions {'version': '3.0', 'spec_version': '2.4'}, 'http_version': '1.1', 'server': ('127.0.0.1', 8000), '3. Environment / path errors	

Evidence of AI Usage

This section includes evidence of significant prompts and responses used or generated through the AI tool. It should provide a clear understanding of the extent to which the AI tool was used in the assignment. Evidence may be attached via screenshots or text.

```
RequestResponseCycle.receive of
<uvicorn.protocols.http.h11_impl.RequestResponseCycle object at
0x146673640>>
|           |           |   L{'type': 'http', 'asgi': {'version': '3.0',
'spec_version': '2.4'}, 'http_version': '1.1', 'server': ('127.0.0.1', 8000),
'cl...
```

Thought for 4 seconds >

Nice — here’s a focused, prioritized **list of common causes** for that Starlette/FastAPI route-dispatch traceback plus the exact quick checks to find the real error (the traceback *you didn’t paste* lives below those frames).

Common causes (ranked by how often they show up)

1. **Import-time exception**
 - Something raised while importing your app/module (model load, bad import, missing file). App appears to register routes but fails later.
2. **Model or resource load failure**
 - Loading a ML model, large file, or native library (CUDA/PyTorch) at import time or first request fails.
3. **Blocking CPU work inside `async def`**
 - Heavy image processing, model inference, or `time.sleep()` performed synchronously in an async route blocks the loop or times out.
4. **Wrong handler signature / parameter mismatch**

Nice — here's a focused, prioritized **list of common causes** for that Starlette/FastAPI route-dispatch traceback plus the exact quick checks to find the real error (the traceback *you didn't paste* lives below those frames).

1. Import-time exception

2. Model or resource load failure

3. Blocking CPU work inside `async def`



— Rare, but can cause connection-level errors that appear in call stack.

Quick, exact steps to find the real error (do these now)

1. Run the app so you see the full traceback

```
uvicorn main:app --reload --port 8000
```

— watch the console when you hit the failing endpoint. The *real* exception message is at the bottom of the printed traceback.
2. Reproduce locally with a minimal request

```
curl -F "file=@/path/to/sample.jpg" http://127.0.0.1:8000/analyze-xray
```

or `http POST :8000/analyze-xray file=@/path/to/sample.jpg` (httpie)
3. Check import-time errors quickly:

```
python -c "import main; print('import ok')"
```

— if this raises, fix the import error first.
4. Increase logging & catch tracebacks
Add at top of app:

RadAssistIQ – Intelligent Chest X-Ray Diagnostic Assistant to Read Smarter and Diagnose Faster

An AI-powered radiology assistant that analyses chest X-rays, identifies potential diagnoses, and explains findings through clinically grounded reasoning.

1st Surya Chandra Raju Kurapati

MSc in Data Analytics

National College of Ireland

Dublin, Ireland

x23396920@student.ncirl.ie

Abstract—RadAssistIQ is an intelligent assistant developed within the healthcare domain for the Radiology department, with potential extension to general physicians. It combines Convolutional Neural Network and Large Language Model to help radiologists interpret chest X-rays with accuracy and efficiency. Key AI concepts such as Deep Learning, Explainable AI (XAI), and Generative AI are applied to the product development to classify several thoracic conditions like pneumonia, pleural effusion, pneumothorax, infiltration, consolidation, and hernia based on X-ray image patterns and provide clinical reasoning for each diagnosis. Each diagnosis is accompanied by a likelihood probability score representing the model's confidence in its prediction.

The model achieved an accuracy of 81.5% in identifying diagnosis from X-rays with the clinical reasoning, enabling RadAssistIQ to deliver reliable, quick responses within 10 seconds, and explainable insights that help radiologists read X-rays faster, detect thoracic abnormalities more confidently, and reduce their diagnostic workload. This project demonstrates how Artificial Intelligence technology and healthcare radiology can work hand-in-hand in improving workflow efficiency, support diagnostic decision-making and contribute to a better patient care.

Index Terms—Healthcare, Radiology, Chest X-ray, Convolutional Neural Network, Generative AI, Explainable AI, Large Language Model, DenseNet121, gemini-2.5-pro, Clinical Decision Support, Thoracic Disease Detection

I. INTRODUCTION

Chest X-rays (CXRs) are an important diagnostic tool in radiology department for the initial

assessment of various pulmonary and cardiac conditions. However, interpreting CXRs can be challenging as subtle abnormalities are often overlooked due to overlapping anatomy, poor image quality, or differences in patient positioning. These difficulties combined with the high patient volumes can result into inconsistent interpretations, diagnostic delays, and increase radiologist workloads.

The challenges motivated the design of RadAssistIQ, developed to support radiologists during early-stage diagnosis by offering fast, accurate, quantifiable, and explainable insights. Figure 1 illustrates how the product fits within the healthcare domain integrated with key technologies used, including CNN for image understanding and LLM for reasoning and explanation. Built by the principles of Explainable AI, the product ensures transparent and interpretable predictions rather than purely algorithmic aligning with the clinical standards expected in radiology.

As illustrated in Figure 2, radiologists interact with this product through a Fast API endpoint by submitting an X-ray ID and query. In the backend, Pillow library is used for X-ray image processing and utilizes DenseNet121 CNN to extract features and classify thoracic conditions with a probability score. These responses are then fed to gemini-1.5-pro LLM that translates algorithmic predictions

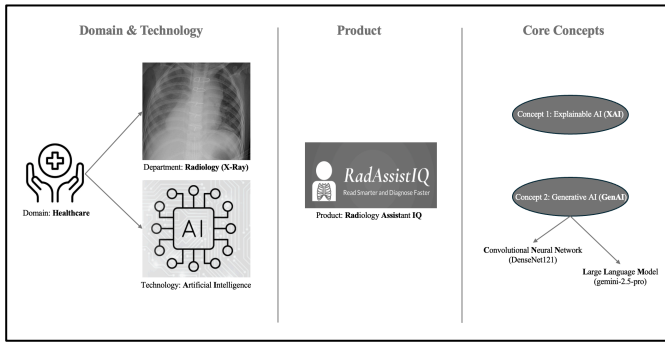


Fig. 1. Domain and Technology overview of RadAssistIQ

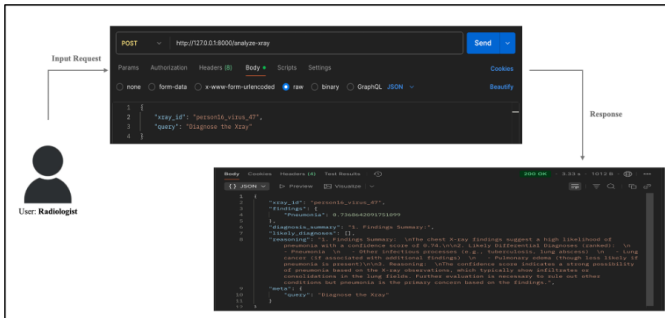


Fig. 2. Interaction workflow between Radiologist and RadAssistIQ product

into medically relevant reasoning.

II. RELATED WORK

Research shows that radiology operates as a fundamental healthcare service because chest imaging procedures have become essential for medical diagnosis. In the United States alone, chest imaging utilization in emergency departments increased by 173% between 1994 and 2015, while chest CT usage rose by 5941.8% per 1000 emergency visits during the same period reflecting the growing clinical dependence on radiologic evaluation for pulmonary and cardiac diseases [1]. Chest X-rays remain one of the frequently performed imaging procedure globally in identifying diagnoses like pneumonia, pleural effusion, and pneumothorax.

Although chest x-ray procedure showcases the diagnostic importance but remain challenging during interpretations leading to errors. Analyses show that underreading alone accounts for about

42% of radiology errors, followed by satisfaction-of-search errors at 22% and faculty reasoning errors at 9%, showing how perception and cognitive factors influence diagnostic accuracy [2]. A recent study also says that radiologists who made diagnostic errors had an average daily workload of 121% showcasing a strong association between overproduction and the likelihood of errors [3]. These pressures are amplified in busy clinical environments where radiologists often work through hundreds of scans during single shift.

To overcome these limitations, recent advances have shown strong potential in automating and assisting radiological workflows using Artificial Intelligence. Deep learning-based approach such as DenseNet121 CNN showcased reliability in pneumonia detection with reported accuracy rate of 97.3% on chest x-ray datasets [6], and a large Emergency Department study further demonstrated that these Deep Learning techniques delivered an AUC of 0.95 in detecting clinical abnormalities on chest radiographs [4]. However, Trust and Interpretability have become a question in these AI based approaches which have led to the development of Explainable AI techniques, where research says that radiologists prefer visual saliency maps and clear explanatory outputs that support clinical reasoning over human-centered evaluations [5]. Meanwhile, Large Language Models have performed extremely well in diagnostic reasoning, where a recent random trial report says that clinicians using LLM assistance achieved 71.4% diagnostic reasoning accuracy compared to 42.6% with conventional resources demonstrating a 27.5% improvement in accuracy without increasing the time required per case [7].

Taking into consideration all the above context, RadAssistIQ was developed as an assistant to help radiologists scan chest X-rays quickly and accurately. While the product uses DenseNet121 CNN to detect thoracic diagnosis, also integrates gemini-2.5-pro LLM model to generate additional clinical reasoning that supports the detected diagnosis. Because it follows Explainable AI principles, the reasoning behind each response is visible to the user explaining how it arrived at each

conclusion instead of just a black box output.

III. ETHICS & CONSIDERATION:

The ethical and regulatory framework of RadAssistIQ is rooted in the principles of data protection, fairness, and transparency as outlined by the General Data Protection Regulation and EU AI Act. Since the product processes healthcare chest X-ray data, it adheres to Article 5 ensuring anonymized images with identifiable metadata is read before model inference. In compliance with Article 9 the product is deployed only for diagnostic assistance and not for automated decision-making without human oversight. Configuration data and API keys are secured in Mongo DB following Article 32 making the access controlled. The transparency principle under Article 12 and Recital 60 is addressed through the explainable feature of the LLM gemini-2.5-pro which provides reasoning for each AI-generated interpretation allowing clinicians to understand the basis of diagnostic suggestions. Furthermore, the system supports Article 22 by preventing any unsupervised clinical conclusions by keeping radiologist in the decision loop. Aligning to these GDPR principles, RadAssistIQ follows ethical AI use ensuring data privacy, human accountability and interpretability for its clinical adoption.

IV. METHODOLOGY

A. Data Collection

For this study, Chest X-ray images were obtained from a publicly available Kaggle dataset. This dataset consists of over 5000 X-ray images stored in jpeg format distributed into train, test, and validation subsets. These images are used for training and evaluating the DenseNet121 CNN model. Specifically, the training set is used for learning visual features and disease representations, the validation set is used for model optimization, and the test set for evaluating diagnostic accuracy and generalization of any unseen data.

Dataset: [Chest X-ray Images Kaggle dataset](#)

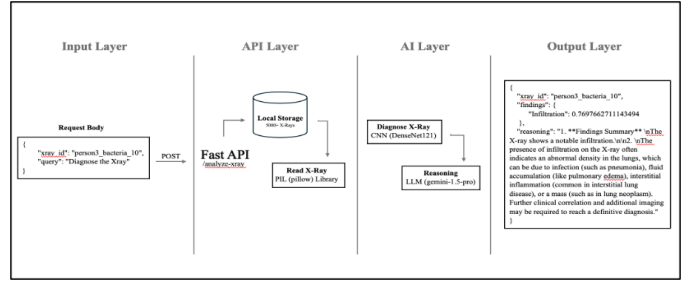


Fig. 3. Four layered Architecture of RadAssistIQ

B. Architecture

As illustrated in Figure 3, RadAssistIQ is designed as a four-layer architecture from user input to LLM generated diagnostic reasoning. The Input layer acts as the entry point where radiologists or clinicians pass a chest X-ray ID and query as request body in json format to the POST API endpoint. This step ensures secure data transfer and initiates the diagnostic workflow. Next, the API layer is built using Fast API that handles the incoming request through the endpoint /analyze-xray. Holding the X-ray ID, system retrieves the corresponding image from the local storage, preprocesses it using Pillow library where resizing and normalization is performed, and validates the input before forwarding it to the LLM model. This layer acts as a bridge between the user and backend.

AI layer serves as the core layer for the analytical and reasoning tasks. This layer integrates DenseNet121, a CNN implemented using PyTorch for image analysis and disease classification across 14 thoracic conditions like Pneumonia and Effusion. The generated diagnosis prediction along with the score is then fed to gemini-2.5-pro LLM model which interprets the results and produces an explainable clinical reasoning. Finally, the Output layer compiles all generated responses into a JSON format comprising both model findings and clinical reasoning and delivers it back to the user. This multi-layered architecture enables RadAssistIQ to integrate computer vision with natural language reasoning and generate a transparent, efficient,

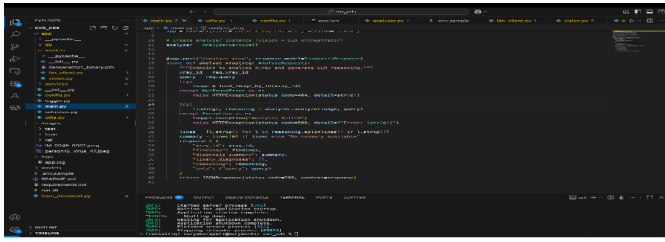


Fig. 4. Implementation of Code

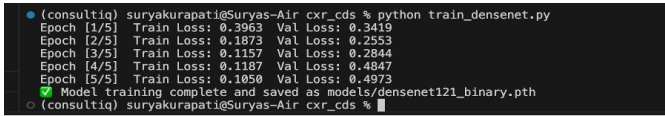


Fig. 5. Densenet121 CNN model training over 300 sample X-rays

and clinically interpretable diagnostic assistant.

C. Code Implementation

The product application is implemented technically in Python using FastAPI following Object Oriented Programming structure as shown in figure 4. The core of the system lies in vision.py file which implements a DenseNet121 CNN model from TorchVision package to analyze chest X-ray images and predict diagnoses based on learned visual patterns. The model uses a pre-trained weight file created as densenet121_binary.pth using an ad hoc script train_densenet.py as shown in figure 5, which was trained over 300 chest X-rays collected from Kaggle. The llm_clinet.py module integrates gemini-2.5-pro LLM that interprets DenseNet121 outputs and generates clinically relevant diagnostic reasoning in natural language simulating how a radiologist describes the findings.

These modules are tightly coupled and called through the AnalyzerService class in the services/ directory which acts as the orchestrator between the vision model and LLM. The application main entry point is main.py file that defines FastAPI routes for accepting user input request body and returning JSON formatted diagnostic results. In addition to these core modules, there are supporting modules like config.py built for managing application settings, logger.py for logging the run flow

and error tracking using Loguru package. The schemas.py file defines input and output data models using Pydantic and utils.py provides helper functions for image processing and file handling.

This technical implementation flow with OOPs concepts and distinct folders for models, services, logs, and images ensures a clean workflow, simplifies debugging, and provides scope for any future enhancements such as integration of patient electronic medical records (EMR) for personalized diagnostic reasoning.

D. Tools and Technology

Table 1 outlines the key tools and technologies used throughout the development and evaluation of RadAssistIQ.

V. EVALUATION

As mentioned in the data collection section, over 300 chest X-ray images from train and validation subsets are used to train the DenseNet121 CNN model using train_densenet.py script. Later, the model performance was assessed on a dataset of 100 images from the test folder evaluating normal and pneumonia affected lungs. The model achieved an overall accuracy of 81.5% correctly classifying 68 normal and 95 pneumonia cases indicating a reliable confidence in product usage. To support the results, the classification report is extracted as shown in Table 2 highlighting string performance metrics with the Normal class achieving a high precision of 0.93 and the Pneumonia class showcasing strong recall of 0.95 leading to F1-scores of 0.78 and 0.83 respectively. Furthermore, the macro and weighted averages remain consistent across precision, recall, and F1-score suggesting balanced behavior without significant class bias. The confusion matrix in Table 3 supports these results further by illustrating the misclassifications were relatively limited specially the pneumonia ones. Overall, these evaluation metrics confirm that the DenseNet121 model integrated in RadAssistIQ product performs reliable well and can effectively support radiologists in identifying thoracic cases.

TABLE I
TOOLS AND TECHNOLOGIES

Category	Tool / Technology	Purpose
Framework	FastAPI	Web framework for building RESTful APIs.
Image Processing	Pillow (PIL)	Used for reading and preprocessing chest X-ray images
Vision Model	DenseNet121	CNN model for detecting thoracic diseases from X-rays
LLM	gemini-2.5-pro	LLM used for clinical reasoning
Database	MongoDB	Stores configuration data and API credentials

TABLE II
CLASSIFICATION REPORT

Class	Precision	Recall	F1-score
Normal	0.9315	0.68	0.7861
Pneumonia	0.7480	0.95	0.8370
Accuracy	0.815	0.815	0.815
Macro Avg	0.8397	0.815	0.8116
Weighted Avg	0.8397	0.815	0.8116

TABLE III
CONFUSION MATRIX

Actual / Predicted	Normal	Pneumonia
Normal	68	32
Pneumonia	5	95

VI. FUTURE WORK

RadAssistIQ is built as a minimum valuable product with baseline AI technical and healthcare domain integrations. However, the product can be enhanced both functionally and technically further to make it more powerful and intelligent radiology support tool. Functionally, it can be integrated with the patient's Electronic Medical Records (EMR) collected from healthcare organization's database to provide additional context such as patient history, symptoms, social history, any previous imaging data to LLM for more personalized and context-aware insights. Technically, the DenseNet121 CNN model can be trained and fine-tuned on larger and diverse datasets covering thoracic, pulmonary, and cardiac conditions to expand its diagnostic capability beyond pneumonia and limited thoracic

conditions. Incorporating advanced Large Language Models such as OpenAI gpt-4 models improve accuracy. As part of future phase, asynchronous API call integration can be implemented that allows multiple users to hit the application at once. In terms of deployment, the entire code can be packaged into a docker container and deployed on cloud service such as AWS Kubernetes cluster pods ensuring scalable and cost optimized delivery. Lastly, a web interface to the user can be built using next.js that provides a chatbot-style experience enabling radiologists and clinicians to engage with RadAssistIQ easily for diagnostic support and real-time query responses.

VII. CONCLUSION

RadAssistIQ was designed and developed from the gaps found in Radiology department such as increased radiologist workloads, frequent perceptual and cognitive errors, and the growing need of interpretable AI. The product unites Deep Learning with Explainable AI and Generative Reasoning to create an intelligent medical assistant for radiologists. This product enables radiology professionals and clinicians to fetch the diagnostic summary of patient's X-ray image within 10 seconds and proves to be reliable with an accuracy of 81.5% resulting in reduced patient care duration and faster and better treatment outcomes. The product reliably supports the early stages of CXR interpretation without removing the radiologist from the decision-making process. Incorporation of Explainable AI principles and ethical considerations ensures that the product

remains transparent, interpretable, and aligned with clinical expectations. While the product is still a minimum valuable product, it provides a roadmap for future enhancements such as EMR integration, fine-tuning of DenseNet121 CNN model, usage of gpt-4 LLM, scalable deployment on cloud. Overall, the project highlights how Artificial Intelligence can be used responsibly to enhance radiology workflows rather than replace human expertise enhancing patient care delivery.

Code Availability: The complete source code and implementation details for this project is available at [GitHub repository](#).

REFERENCES

- [1] J. H. Chung, R. Duszak Jr., J. Hemingway, D. R. Hughes, and A. B. Rosenkrantz, "Increasing utilization of chest imaging in US emergency departments from 1994 to 2015," *J. Am. Coll. Radiol.*, vol. 16, no. 5, pp. 674–682, 2019, doi: [10.1016/j.jacr.2018.11.011](#).
- [2] F. Pesapane et al., "Errors in Radiology: A Standard Review," *J. Clin. Med.*, vol. 13, no. 15, p. 4306, 2024, doi: [10.3390/jcm13154306](#).
- [3] Ö. Kasalak, H. Alnahwi, R. Toxopeus, J. P. Pennings, D. Yakar, and T. C. Kwee, "Work overload and diagnostic errors in radiology," *Eur. J. Radiol.*, vol. 167, p. 111032, 2023, doi: [10.1016/j.ejrad.2023.111032](#).
- [4] E. J. Hwang et al., "Deep learning for chest radiograph diagnosis in the emergency department," *Radiology*, vol. 293, no. 3, pp. 573–580, 2019, doi: [10.1148/radiol.2019191225](#).
- [5] E. Ihongbe, S. Fouad, T. F. Mahmoud, A. Rajasekaran, and B. Bhatia, "Evaluating Explainable Artificial Intelligence (XAI) techniques in chest radiology imaging through a human-centered lens," *PLoS One*, vol. 19, no. 10, e0308758, 2024, doi: [10.1371/journal.pone.0308758](#).
- [6] T. S. Arulananth, S. W. Prakash, R. K. Ayyasamy, V. P. Kavitha, P. G. Kuppusamy, and P. Chinnasamy, "Classification of Paediatric Pneumonia Using Modified DenseNet-121 Deep-Learning Model," *IEEE Access*, vol. 12, pp. 35716–35727, 2024, doi: [10.1109/ACCESS.2024.3371151](#).
- [7] I. A. Qazi, A. Ali, A. U. Khawaja, M. J. Akhtar, A. Z. Sheikh, and M. H. Alizai, "AI-literacy training enhances physician–LLM diagnostic collaboration in a resource-limited setting: a randomized controlled trial," *medRxiv*, 2025, doi: [10.1101/2025.06.06.25329104](#).